

Control_SE_board

Установка и запуск

1. Клонировать репозиторий:

```
git clone https://github.com/ваш-логин/Control_SE_board.git
cd Control_SE_board
```

2. Создайте и активируйте виртуальное окружение (рекомендуется Python 3.x):

```
python -m venv venv
source venv/bin/activate # (Linux/macOS)
venv\Scripts\activate   # (Windows)
```

3. Установите зависимости:

```
pip install -r requirements.txt
```

4. Запустите приложение (пример для вашего скрипта):

```
python controlboard.py --help
```

Описание

Control_SE_board - это консольное приложение для управления аппаратной платой (Small_Edge) через интерфейс Modbus. Программа позволяет считывать параметры, управлять устройствами (такими как радары, камеры, нагреватели), а также обновлять прошивку и проводить конфигурацию системы.

Общий принцип управления

Программа взаимодействует с устройством по протоколу Modbus через COM-порт. Все команды, выполняемые программой, основаны на структуре запроса Modbus: адрес устройства, функция, адрес регистра и значение (если применимо).

Команды разделены по назначению и типу действия. Каждой команде соответствует словарь, описывающий, что она делает, какой тип у неё (TYPE), и какие параметры участвуют в запросе.

Типы команд и их назначение

READ — чтение данных с устройства. Используется для получения текущих значений с датчиков, регистров состояния, напряжений, температур, флагов, версий и других параметров. Выполняется через

функции чтения Modbus: 0x01, 0x02, 0x03, 0x04.

WRITE — запись конфигурационных параметров или управление устройством с параметром (управление двигателями - указывается кол-во шагов и направление) Используется для настройки устройства: пороги температур, гистерезисы, лимиты тока и напряжения и т.д. Чтение может использоваться отдельно, а запись позволяет эти параметры изменить.

CONTROL — управление устройствами Команды, которые изменяют состояния оборудования: включение/выключение радара, камер, GPS, светодиодов, ПК и т.д. В основном отправляются без зависимости от значений пользователя.

TEST — тестовые команды Используются для отладки и проверки работы определённых функций платы, например: watchdog таймера или записи во Flash.

UPDATE — обновление прошивки (автоматизированная) Предназначено для переключения микроконтроллера в режим загрузки или выполнения других операций, связанных с обновлением ПО.

Как получить список доступных команд?

Чтобы посмотреть все команды и их назначение, запусти:

```
python controlboard.py --help или -h
```

Это откроет общую справку по всем типам команд: `read`, `write`, `control`, `test`, `update`, `my_bytes`, `util`.

Можно указать тип, чтобы увидеть команды только из этой категории:

```
python controlboard.py read -h
python controlboard.py write -h
python controlboard.py control -h
python controlboard.py test -h
python controlboard.py update -h
python controlboard.py my_bytes -h
python controlboard.py util -h
```

Каждый такой вызов покажет список доступных подкоманд и краткие описания к ним. Это удобно, если ты хочешь быстро найти нужную команду в рамках конкретной задачи.

Пример:

```
python controlboard.py read -h
```

покажет только команды чтения.

\

Команды управления (control)

- `led1_on` - Включение светодиода LED1 (БЦО: H14 - SE_V1, H2 - SE_V2).

Пример команды (COM-порт должен соответствовать вашему):

```
python controlboard.py control -p COM3 led1_on
```

Примечание: остальные команды запрашиваются таким же образом, пример не будем дублировать.

- `led1_1hz` - Мигание светодиодом LED1 с частотой 1 Гц (БЦО: H14 - SE_V1, H2 - SE_V2).
- `led1_2hz` - Мигание светодиодом LED1 с частотой 2 Гц (БЦО: H14 - SE_V1, H2 - SE_V2).
- `led1_1s` - Мигание светодиодом LED1 1 секунду (БЦО: H14 - SE_V1, H2 - SE_V2).
- `led1_2s` - Мигание светодиодом LED1 2 секунды (БЦО: H14 - SE_V1, H2 - SE_V2).
- `led1_off` - ВыКЛючение светодиода LED1 (БЦО: H14 - SE_V1, H2 - SE_V2).
- `led2_on` - Включение светодиода LED2 (БЦО: H15 - SE_V1, H3 - SE_V2).
- `led2_1hz` - Мигание светодиодом LED2 с частотой 1 Гц (БЦО: H15 - SE_V1, H3 - SE_V2).
- `led2_2hz` - Мигание светодиодом LED2 с частотой 2 Гц (БЦО: H15 - SE_V1, H3 - SE_V2).
- `led2_1s` - Мигание светодиодом LED2 1 секунду (БЦО: H15 - SE_V1, H3 - SE_V2).
- `led2_2s` - Мигание светодиодом LED2 2 секунды (БЦО: H15 - SE_V1, H3 - SE_V2).
- `led2_off` - ВыКЛючение светодиода LED2 (БЦО: H15 - SE_V1, H3 - SE_V2).
- `leds_on` - ВКЛючение индикации на плате.
- `leds_off` - ВыКЛючение индикации на плате.
- `radar_on` - вкл РАДАР.
- `radar_off` - выкл РАДАР.
- `radar_on_2` - вкл РАДАР (альтернативная команда).
- `radar_off_2` - выкл РАДАР (альтернативная команда).
- `cam_on` - вкл КАМЕРУ.
- `cam_off` - выкл КАМЕРУ.
- `cam_on_2` - вкл КАМЕРУ (альтернативная команда).
- `cam_off_2` - выкл КАМЕРУ (альтернативная команда).
- `gps_on` - вкл GPS.
- `gps_off` - выкл GPS.
- `gps_on_2` - вкл GPS (альтернативная команда).
- `gps_off_2` - выкл GPS (альтернативная команда).
- `pc_on` - вкл ПК.
- `pc_off` - выкл ПК.
- `pc_wdt_reset` - Сброс таймера Watchdog ПК (Сбрасывает на 2 минуты).
- `reset` - команда сброса МК, обычно для перехода в режим обновления ПО.
- `params_save` - Сохранение конфигурационных параметров во Flash.
- `params_reset` - Сброс конфигурационных параметров и запись во Flash.
- `freez` - Включение freez mode (остановка WDT ПК).
- `unfreez` - Выключение freez mode (возобновление работы WDT ПК).
- `heat1_on` - вкл НАГРЕВАТЕЛЬ 1 (X14 - SE_V1 или 3-4 пины X2 - SE_V2). Подает постоянный уровень +12 В. Также работает и с командой `write`.
- `heat1_off` - выкл НАГРЕВАТЕЛЬ 1. Также работает и с командой `write`.

- `heat2_on` - вкл НАГРЕВАТЕЛЬ 2 (X19 - SE_V1 или 1-2 пины X2 - SE_V2). Подает постоянный уровень +12 В. Также работает и с командой `write`.
- `heat2_off` - выкл НАГРЕВАТЕЛЬ 2. Также работает и с командой `write`.

Команды чтения (`read`)

- `temp` - Чтение датчика температуры напрямую.

Пример запроса (COM-порт должен соответствовать вашему):

```
python controlboard.py read -p COM3 temp
```

Примечание: остальные команды отправляются таким же образом, пример не будем дублировать.

- `version_request` - запрос версии, в ответ 30 04 02 20 00 (и 2 байта crc16) - перед правками было 30 04 02 14 00.
- `coils` - Чтение цепей питания.
- `states` - Чтение состояний флагов.
- `pc_wdt` - Чтение Watchdog таймера ПК.
- `voltage` - Чтение напряжения на входе, зафиксированное при последнем обновлении данных с датчика.
- `temperature` - Чтение температуры, зафиксированной при последнем обновлении данных с датчика.
- `current` - Чтение тока на входе, зафиксированного при последнем обновлении данных с датчика.
- `start_temp` - Чтение лимитов стартовой температуры (конфигурационные параметры работы прибора).
- `work_temp` - Чтение лимитов рабочей температуры (конфигурационные параметры работы прибора).
- `pre_temp` - Чтение параметра температуры преднагрева (конфигурационный параметр работы прибора).
- `hyst` - Чтение гистерезиса переключения нагревателей (конфигурационный параметр работы прибора).
- `voltage_limits` - Чтение диапазона нормальной работы по напряжению (конфигурационные параметры работы прибора).
- `current_limits` - Чтение диапазона нормальной работы прибора по току (конфигурационные параметры работы прибора).
- `firmware_version` - Чтение версии прошивки, установленной при первом старте прибора (читается из Flash-памяти).
- `heat_temp` - Чтение установленных границ срабатывания нагревателей (конфигурационный параметр работы прибора).
- `serial_number` - Чтение серийного номера микроконтроллера STM32.
- `with_fan` - Чтение параметра работы FAN (конфигурационный параметр работы прибора).
- `ups_conf` - Чтение параметра работы UPS (конфигурационный параметр работы прибора).
- `bat_low_limit` - Чтение установленного порога низкого заряда батареи (конфигурационный параметр работы прибора).
- `timer` - Чтение счетчика времени работы платы (время не совсем точное).

- `accum` - Чтение саккумулированных значений потребления (в данную функцию включена функция "timer").
- `humidity` - Чтение значения влажности с датчика влажности.
- `pressure` - Чтение значения датчика давления.
- `in_v` - Запрос входного напряжения прямо с датчика.
- `in_v_high_lim` - Запрос лимита перенапряжения входного питания прямо с датчика.
- `in_v_low_lim` - Запрос лимита пониженного напряжения входного питания прямо с датчика.
- `in_meter` - Запрос данных счетчика в кВт входного питания прямо с датчика.
- `in_v_shunt` - Запрос падения напряжения на входном шунте прямо с датчика/
- `in_i` - Запрос входного тока прямо с датчика.
- `in_i_lim` - Запрос лимита установленного по току прямо с датчика.
- `in_p` - Запрос входной мощности прямо с датчика.
- `in_p_lim` - Запрос лимита установленного по мощности прямо с датчика.
- `pc_v` - Запрос напряжения ПК прямо с датчика.
- `pc_v_high_lim` - Запрос лимита перенапряжения питания ПК прямо с датчика.
- `pc_v_low_lim` - Запрос лимита пониженного напряжения питания ПК прямо с датчика.
- `pc_meter` - Запрос данных счетчика в кВт питания ПК прямо с датчика.
- `pc_v_shunt` - Запрос падения напряжения на шунте ПК прямо с датчика.
- `pc_i` - Запрос тока ПК прямо с датчика.
- `pc_i_lim` - Запрос лимита установленного по току для ПК прямо с датчика.
- `pc_p` - Запрос мощности потребляемой ПК прямо с датчика.
- `pc_p_lim` - Запрос лимита установленного по мощности для ПК прямо с датчика.
- `ra_v` - Запрос напряжения радара прямо с датчика.
- `ra_v_high_lim` - Запрос лимита перенапряжения питания радара прямо с датчика.
- `ra_v_low_lim` - Запрос лимита пониженного напряжения питания радара прямо с датчика.
- `ra_meter` - Запрос данных счетчика в кВт питания радара прямо с датчика.
- `ra_v_shunt` - Запрос падения напряжения на шунте радара прямо с датчика.
- `ra_i` - Запрос тока радара прямо с датчика.
- `ra_i_lim` - Запрос лимита установленного по току радара прямо с датчика.
- `ra_p` - Запрос мощности потребляемой радаром прямо с датчика.
- `ra_p_lim` - Запрос лимита установленного по мощности радара прямо с датчика.
- `sl_v` - Запрос напряжения питания прожектора прямо с датчика.
- `sl_v_high_lim` - Запрос лимита перенапряжения питания прожектора прямо с датчика.
- `sl_v_low_lim` - Запрос лимита пониженного напряжения питания прожектора прямо с датчика.
- `sl_meter` - Запрос данных счетчика в кВт питания прожектора прямо с датчика.
- `sl_v_shunt` - Запрос падения напряжения на шунте прожектора прямо с датчика.
- `sl_i` - Запрос тока прожектора прямо с датчика.
- `sl_i_lim` - Запрос лимита установленного по току для прожектора прямо с датчика.
- `sl_p` - Запрос мощности потребляемой прожектором прямо с датчика.
- `sl_p_lim` - Запрос лимита установленного по мощности для прожектора прямо с датчика.
- `tech_data` - Запрос технической информации, записанной при прошивке (даты записи, версии и т.д.).
- `imu` - Чтение данных акселерометра, гироскопа и углов наклона.
- `rtc` - Чтение даты и времени с RTC микроконтроллера.
- `frame_mult` - Чтение параметров умножения тактового сигнала для прожектора (и внутренний и внешний).

- `frame_state` - Чтение состояния в процессе работы тактовых сигналов для прожектора (и внутренний и внешний).
- `autoheat_mode` - Чтение режима автонагрева.

Команды записи (`write`)

Нагреватели

- `heat1_on` - вкл НАГРЕВАТЕЛЬ 1 (X14 - SE_V1 или 3-4 пины X2 - SE_V2). Данная команда без указания значения подает постоянный уровень +12 В.
- `heat1_on -v 1` - вкл НАГРЕВАТЕЛЬ 1 в режиме ШИМ с длительностью импульса 110 мкс, при значении 2 - 120 мкс и т.д.
- `heat1_off` - выкл НАГРЕВАТЕЛЬ 1. Также работает и с командой `control`.
- `heat2_on` - вкл НАГРЕВАТЕЛЬ 2 (X19 - SE_V1 или 1-2 пины X2 - SE_V2) - см. нагреватель 1.
- `heat2_off` - выкл НАГРЕВАТЕЛЬ 2 - см. нагреватель 1.

Примеры команд (COM-порт должен соответствовать вашему):

```
python controlboard.py write -p COM3 heat1_on
```

```
python controlboard.py write -p COM3 heat1_on -v 1
```

```
python controlboard.py write -p COM3 heat1_off
```

Моторы

- `focus` - Управление мотором фокуса.
- `zoom` - Управление мотором зума.
- `diaph` - Управление мотором диафрагмы.
- `cam_filter` - Управление мотором фильтра камеры.

Пример

```
python controlboard.py write -p COM3 focus -v ">120"
```

Описание команды: Мотор фокуса повернуть по часовой стрелке (смотря на вал) на 120 шагов. Данная команда многосоставная под капотом - отправляет несколько команд при больших значениях. При больших значениях отправляется несколько команд микроконтроллеру.

Примечание: использование символики типа ">" в значении обязывает возводить значение в кавычки.

Далее идут команды по записи конфигурационных параметров. Примечание: Значения записываются в глобальные переменные, т.е. после перезагрузки примут дефолтные значения или

значения записанные по команде из раздела **control** - **save_params**. Значения имеют ограничения только в самой прошивке, если вводится недопустимое, то возвращается соответствующая ошибка.

- **start_low_temp** - Записываем нижнюю границу стартовой температуры (default: 0 [°C]).

Пример:

```
python controlboard.py write -p COM3 start_low_temp -v -3
```

можно указать необходимое значение

```
python controlboard.py write -p COM3 start_low_temp -v default
```

а можно дефолтное, просто вписав значение **default**

- **start_high_temp** - Записываем верхнюю границу стартовой температуры (default: 70 [°C]).
- **work_low_temp** - Записываем нижнюю границу рабочей температуры (default: -5 [°C]).
- **work_high_temp** - Записываем верхнюю границу рабочей температуры (default: 85 [°C]).
- **hyst** - Записываем значение гистерезиса для работы нагревателей (default: 2 [°C]).
- **max_volt** - Записываем значение максимального рабочего напряжения (default: 15000 [mV]).
- **min_volt** - Записываем значение минимального рабочего напряжения (default: 10500 [mV]).
- **max_current** - Записываем значение максимального рабочего тока (default: 20000 [mA]).
- **min_current** - Записываем значение минимального рабочего тока (default: 50 [mA]).
- **heater1_temp** - Записываем значение предела работы нагревателя 1 (default: 5 [°C]).
- **heater2_temp** - Записываем значение предела работы нагревателя 2 (default: 5 [°C]).
- **frame_inn_mult** - Запись параметра умножения тактового сигнала для прожектора (внутренний канал), где значение может быть 1...10 (default: 1).
- **frame_ext_mult** - Запись параметра умножения тактового сигнала для прожектора (внешний канал), где значение может быть 1...10 (default: 1).
- **autoheat_mode** - Установка режима автонагрева (default: 0 - HEAT_SET_BOTH - оба канала нагревателя работают в режиме ВКЛ-ВЫКЛ).

Режимы автонагрева Пример установки режима автонагрева:

```
python controlboard.py write -p COM3 autoheat_mode -v 1
```

Существуют следующие режимы:

- **0** - HEAT_SET_BOTH - оба канала питания нагревателей работают в режиме ВКЛ-ВЫКЛ
- **1** - HEAT_PWM1_SET2 - первый канал работает в подобранном режиме ШИМ, второй - ВКЛ-ВЫКЛ
- **2** - HEAT_SET1_PWM2 - первый канал работает в режиме ВКЛ-ВЫКЛ, второй - в подобранном режиме ШИМ
- **3** - HEAT_PWM_BOTH - оба канала питания нагревателей работают в подобранном режиме ШИМ

- 4 - HEAT_SET_1 - только первый канал работает в режиме ВКЛ-ВЫКЛ
- 5 - HEAT_SET_2 - только второй канал работает в режиме ВКЛ-ВЫКЛ
- 6 - HEAT_PWM_1 - только первый канал работает в подобранном режиме ШИМ
- 7 - HEAT_PWM_2 - только второй канал работает в подобранном режиме ШИМ Пояснение: т.е. при пересечении установленных температурных границ автоматически включаются нагреватели, согласно установленному режиму.

Установка времени (RTC) при необходимости ручной настройки.

На данный момент у нас всего два варианта установки даты и времени: локальное время с операционной системы или полностью ручной ввод. Данные тремя командами записываются прямо в RTC. Проверка входных данных присутствует. Примеры см. ниже.

Пример установки локального времени:

```
python controlboard.py write -p COM3 rtc
```

Пример ручного варианта:

```
python controlboard.py write -p COM3 rtc -v "01.01.25 17:03:00"
```

, где значение в формате "DD.MM.YY hh:mm:ss".

Примечание: Формат данных бинарный, в отличие от дат записанных в технические параметры. Даты, такие как: дата установки бутлоадера, дата установки заводской прошивки и апдейта, - хранятся и передаются в формате BCD (Binary-Coded Decimal), чтобы читались в ячейках Flash-памяти.

Команда обновления прошивки (update)

Перед запуском должны быть выполнены соответствующие инструкции - [см. тут](#). Т.к. на плату предварительно должен быть установлен Bootloader.

Пример команды обновления:

```
python controlboard.py update -p COM3 -f "firmware.hex"
```

где аргумент `-f` (или `--file`) должен содержать полный путь к файлу прошивки, если он не находится в корне программы `controlboard.py`.

Также есть возможность использовать дополнительные аргументы:

`--date_b` - дата установки бутлоадера в формате `mm.dd.yy` (по дефолту, если не вводить значение, устанавливается сегодняшняя дата).

`--ver_f` - версия установленной заводской прошивки (по дефолту - `00.00.00`).

`--date_f` - дата установленной заводской прошивки в формате `mm.dd.yy` (по дефолту устанавливается сегодняшняя дата).

`--ver_u` - версия устанавливаемой прошивки апдейта (по дефолту - `00.00.00`).

`--date_u` - дата устанавливаемой прошивки апдейта в формате `mm.dd.yy` (по дефолту устанавливается сегодняшняя дата).

Примечание: Аргументы `--date_b`, `--ver_f`, `--date_f` записываются при первом старте потом можно менять только `--ver_u` и `--date_u` при апдейте. При первом старте бутлоадер и заводская прошивка должны быть предустановлены. Апдейт тоже устанавливается только с предустановленным бутлоадером, т.к. процесс апдейта - взаимодействие с этапами работы бутлоадера.

Пример полной команды при первом старте:

```
python controlboard.py update -p COM3 -f "firmware.hex" --date_b "02.03.25" --  
ver_f "02.00.00" --date_f "04.04.25" --ver_u "02.00.00"
```

Первый старт - имеется ввиду, что прошивки залиты через ПО типа STM32 ST-LINK Utility, но при этом не происходило взаимодействия с прошивкой через терминал, через который можно вручную проделать ту же процедуру.

Отправка произвольных байт (`my_bytes`)

Пример чтения температуры прямо с датчика на плате (для теста):

```
python controlboard.py my_bytes -p COM3 --bytes "30 04 91 00 00 00" --crc True
```

Примечание: по дефолту аргумент `--crc` равен `False`, поэтому при отправки без CRC, просто не используйте этот аргумент

Использование утилит (`util`)

В утилитах у нас будут находиться специальные функции, которые не используются в повседневной работе, а только в определенных ситуациях.

- `back` - откат к заводской прошивке. Пример запуска:

```
python controlboard.py util back -p COM3
```

- `CP2105_RST` - мягкий программный сброс USBtoCOM CP2105.

Пример использования для Windows:

```
python controlboard.py util CP2105_RST
```

Пример использования для Linux:

```
sudo venv/bin/python controlboard.py util CP2105_RST
```

Примечание: конкретизация `venv/bin/python` обязательна, потому что при запуске с `sudo` путь к пакетам берется общей системы и есть вероятность, что нужный модуль не найдется или они не подключатся. На Linux можно в отдельном терминале запустить для отслеживания переподключения:

```
sudo dmesg -wT | grep -i usb
```

Команды тестирования (`test`)

На данный момент тесты не утверждены и многие из них не отработаны - спросить у разработчика.

Тесты для Flash-памяти

Пример использования

```
python controlboard.py test -p COM3 flash
```

Описание:

- в тесте определены значения, на которые мы поменяем имеющиеся - это -11 и 111
- мы меняем диапазон стартовой температуры (даже если забудется его поменять, пока мы не используем данные параметры в работе прошивки)
- сначала читается дефолтный диапазон температур, где мы видим что-то подобное

```
>>> Start tempetarue (config): 0...70 [°C] <<<
```

- потом записывается нижнее значение (пока запись происходит в переменную, т.е. после этой команды, если прервать и сбросить питание, то значение будет дефолтным)

```
>>> [WRITED] Low limit of start temperature (config): -11 [°C] <<<
```

- записывается верхнее значение

```
>>> [WRITED] High limit of start temperature (config): 111 [°C] <<<
```

- дальше снова читается диапазон - убедиться, что значения установились

```
>>> Start tempetarue (config): -11...111 [°C] <<<
```

- следующим этапом происходит сама запись во Flash, для этого существует команда и она как раз отправляется
- происходит ресет микроконтроллера
- потом ожидание когда загрузится прошивка после ресета - 15 секунд
- убеждаемся, что после ресета данные остались

>>> Start tempetarue (config): -11...111 [°C] <<<

- устанавливаем дефолтные значения (это относится ко всем значениям, т.е., если вы хотите проверить плату со специальными параметрами, уже предустановленными - переписать все параметры или провести тестирование вручную)
- убеждаемся - дефолтные значения записаны

>>> Start tempetarue (config): 0...70 [°C] <<<

- дальше по желанию, если требуется убедиться, что дефолтные значения после перезагрузки остаются, - сбросить питание или ресетнуть МК